

Toward an LLM-Powered Operating System: A Natural Language-Based Prototype

Tamer Bahgat Elserwy^{a,*}, Basma E. El-Demerdash^{b,2}

^aMinistry of Education and Higher Education, Doha, 35111, Qatar

^bDepartment of Operations Research and Management, Faculty of Graduate Studies for Statistical Research, Cairo University, Giza, Egypt

ARTICLE INFO

Keywords:

Large Language Model Operating System
AI-Powered OS
Natural Language Interface to the Windows Command
Operating System-based AI

ABSTRACT

The command-line interface (CLI) remains a powerful yet syntactically rigid tool for interacting with operating systems, often posing usability challenges for developers and system administrators. While Large Language Models (LLMs) have advanced high-level user assistance, no practical system has yet enabled direct natural language (NL) control of the native Windows CLI in user mode. This paper presents a fine-tuned LLM prototype that bridges this gap by translating NL instructions into executable Windows commands without relying on agent mediation, GUI simulation, or kernel modifications. Built on a 4-bit quantized LLaMA-3-8B model and adapted via Low-Rank Adaptation (LoRA), the system is trained on a curated dataset of 1,000 NL-to-command pairs sourced from Microsoft documentation. It executes commands securely using Python's subprocess module and incorporates a prompt-engineered safety layer to block or rewrite hazardous inputs. Evaluation on 25 test cases yields a 92% exact syntax match and a BERTScore F1 of 98.91, demonstrating strong semantic fidelity. The model generalizes effectively to unseen instructions, such as generating "netstat -an" from "Monitor network connections". Deployed entirely in user space, the system requires no privileged access or internet connectivity. This work establishes a reproducible pathway toward NL-driven system administration using lightweight, domain-specific LLMs.

1. Introduction

The aspiration to control computers using natural language—without the need for complex syntax and memorized commands—has been a persistent focus in human-computer interaction research. Although contemporary Large Language Models (LLMs) have transformed application-level support, the fundamental interface for power users interacting with the operating system remains rooted in the command line. For system administrators, developers, and advanced users, this imposes an ongoing cognitive challenge: converting high-level intentions into exact, low-level CLI syntax. Recent theoretical frameworks have proposed LLMs as OS kernels or as universal mediators for system calls, yet these ideas are predominantly conceptual and lack practical methods for safe, native command execution—particularly on Windows.

There remains a significant gap: no current system replaces the Windows command interpreter CMD.EXE with a large language model (LLM) operating in user mode. Previous attempts either facilitate interaction via intermediary agent layers (such as AIOS and Cerebrum), mimic GUI activities, or require elevated kernel permissions (such as eBPF). These methods introduce extra layers of abstraction, decrease responsiveness, or jeopardize security, thereby failing to provide a straightforward, direct natural language to command-line interface experience.

To tackle this issue, we present the initial practical user-mode prototype that substitutes CMD.EXE with a customized LLM. This allows end-users to input commands in natural language, such as "Show all running processes," and obtain results returned in natural language. Our contribution comprises three main elements: (1) A secure, lightweight architecture that employs 'subprocess' alongside prompt-engineered safety filters to avert destructive actions; (2) A domain-specific LLaMA-3-8B model, which is effectively fine-tuned using LoRA on a tailored Windows CLI dataset; (3) Empirical evidence demonstrating a 92% exact command accuracy and robust generalization to new tasks, comparable to proprietary models without the need for cloud support.

The rest of this paper is organized as follows: Section 2 offers a critical review of related work on LLM-based OS interfaces and highlights our distinct approach. Section 3 describes our methodology, covering everything from dataset creation to deployment. Section 4 showcases detailed results and error analysis. Finally, Section 6 concludes by discussing implications and future research directions.

2. Related Work

The aspiration to replace the command-line interface (CLI) with natural language (NL) as the primary mode of human-computer interaction has evolved from a speculative vision into a tangible research frontier. While recent advances in Large Language Models (LLMs) have enabled agents to automate complex workflows, the core interaction between users and the operating system—particularly for system administrators, developers, and power users—remains anchored in syntactically rigid, low-level commands. This section critically analyzes the evolution of LLM-powered

*Corresponding author

 t.elserwy1712@education.qa (T.B. Elserwy)

 <https://www.researchgate.net/profile/Tamer-Elserwy> (T.B. Elserwy)

Elserwy)

ORCID(s): 0009-0001-3682-0188 (T.B. Elserwy)

systems, distinguishing between agent-centric frameworks and OS-integrated solutions, and positions our work as the first practical, user-mode prototype that directly bridges NL intent with native Windows CLI execution, leveraging subprocess and prompt engineering for safety and usability.

2.1. Conceptual Foundations: LLMs as the OS Kernel

In 2023, a pivotal conceptual advancement was introduced in this study's vision of the AIOS-Agent ecosystem. It officially aligned traditional OS components with LLM primitives: LLM as the kernel, context window as memory, external storage as the file system, tools as device drivers, and natural language prompts as the system call interface [?]. This framework suggested that LLMs could serve as foundational intelligence for a new kind of operating system, an 'OS with soul,' beyond their role as application-layer assistants. Nevertheless, it remained an abstract analogy, lacking any implementation or mechanism for translating natural language into tangible, secure, low-level system operations.

An additional study expanded on this vision with Prompt-to-OS (P2OS), proposing an LLM layer directly above the traditional OS's system call interface [?]. This concept viewed the LLM as a universal mediator, executing natural language commands and managing low-level operations, potentially replacing CLI/GUI. The architecture even proposed multinational neural databases for persistent memory. However, P2OS also remained a high-level vision, with insufficient technical details on how the LLM would generate, validate, or securely execute the necessary system calls to interface with the kernel. Furthermore, it presumed a deep integration within the OS, rather than functioning as a user-mode layer. A comprehensive survey [?] formalized the evolution into three stages: AI-Powered OS (AI as a tool), AI-Refactored OS (AI embedded in modules), and AI-Driven OS (AI replacing the kernel). This survey confirmed the direction of our research but did not suggest a mechanism to implement the kernel itself.

2.2. Architectural Implementations: Agent Frameworks and OS-Embedded Systems

Recent work has produced several influential agent frameworks that operate as user-space applications on top of the OS, rather than as the OS itself. A landmark example is AIOS (LLM Agent Operating System), a modular daemon providing standardized APIs (e.g., `execute_llm_syscall`) for LLM agents to access models, memory, and tools. AIOS achieves up to 2.1× higher throughput than LangChain or AutoGen through efficient scheduling but remains an agent orchestration layer, not a direct OS interface—users interact with agents, not the OS [?]. Building on AIOS, LSFS (LLM-Based Semantic File System) enables natural language file operations (e.g., "Find all log files from yesterday containing 'ERROR'") by using LLMs and semantic embeddings, outperforming keyword-based search. However, LSFS runs as an application in AIOS, which runs itself on Windows,

without kernel or shell integration [?]. Cerebrum, an AIOS-based SDK and web interface, offers a conversational portal for agent discovery and interaction—foreshadowing direct user-OS dialogue—but still mediates all requests through agents and the AIOS [?]. Other approaches explore deeper integration: Herding LLaMaS demonstrates that LLMs can interpret textual hardware specs and generate low-level OS policies (e.g., memory placement), showing LLMs can reason about system constraints—but targets Linux kernel/driver integration, not user-mode interaction [?]. LiteCUA implements a computer-use agent that controls a desktop via AIOS as an "MCP Server," coming closest to natural language OS control—yet the user still communicates with an agent, not the OS directly [?]. Similarly, OS-Copilot and related systems use LLMs to drive GUI actions (click, type) via simulated input, operating at the graphical layer without touching CLI or system APIs [?]. Finally, Ken explores LLM-generated eBPF programs for in-kernel monitoring—a powerful but privileged, platform-specific (Linux-only), and high-risk approach requiring formal verification [?]. In contrast to all these, this work proposes a direct natural language interface to the OS in user mode—bypassing agent mediation, GUI simulation, and kernel modification—enabling users to command the OS natively through language.

2.3. Memory and State: The Engine of Long-Term Interaction

One significant challenge for natural language systems is the ability to sustain consistent and personalized context. MemGPT introduced a "virtual memory" model that utilizes the LLM's context window like RAM, implementing a two-tier hierarchy system—`recall_storage` (cache) and `archival_storage` (disk) to facilitate long-context reasoning [?]. Although foundational, MemGPT is a library and not an OS-integrated service. MemOS expanded on this concept by creating an entire memory OS with three tiers: Short-Term (STM), Medium-Term (MTM), and Long-Term Persona Memory (LPM), marking the first time a user's persona has been treated as a primary OS resource [?]. Despite its innovative approach, MemOS functions at the agent level rather than the shell level.

2.4. The Command Line: The Unbroken Frontier

Despite progress, the CLI remains unaltered for end-users. Early work like NL2Bash mapped natural language to bash commands using semantic parsing but was limited to a small, curated dataset and Linux environments, lacking robustness to novel queries or state management [?]. Systems like AutoOS and Ken use LLMs for Linux system optimization or eBPF-based kernel extensions [?],[?], but they augment—not replace—the shell. Similarly, AIOS Compiler and AIOS SDK enable command generation for developers [?],[?], but do not offer a direct, seamless natural language interface for end-users.

2.5. Synthesis and Positioning

The literature indicates a distinct progression from conceptual frameworks [?],[?] to agent platforms [?], semantic subsystems [?], and memory architectures [?],[?]. However, a significant gap persists: there is no system that substitutes the Windows command-line interpreter (CMD.EXE) with a fine-tuned LLM as a native, user-mode component. All earlier efforts have considered the LLM as an external agent, parser, or service. Conversely, the prototype proposed in this paper is the first fine-tuned LLM designed to directly replace the Windows CMD in user mode. It interprets natural language commands (e.g., check disk space on drive C). Our prototype parses this intent, understands the resource threshold, generates the precise `wmic logicaldisk get size,freespace,caption` command, and executes them via `subprocess.run()`. The output is returned as natural language, completing the loop. Crucially, our system requires no kernel modifications, privileged access, or eBPF [?]. It incorporates a safety layer using prompt engineering, few-shot examples, and semantic validation to block or rewrite dangerous commands (e.g., `del C: format C:`).

3. Methodology

This section details the end-to-end methodology for fine-tuning a Large Language Model (LLM) to interpret natural language instructions into executable Windows command line interface (CLI) commands. The pipeline encompasses environment setup, dataset preparation, model selection and configuration, supervised fine-tuning, and a multi-faceted evaluation framework designed to assess both syntactic accuracy and semantic correctness of generated commands.

3.1. Environment Setup and Dependencies

The experimental environment was configured using Python 3.10+ with key libraries including Unsloth for efficient model loading and LoRA adaptation, transformers and datasets from Hugging Face for model and data handling, `trl` for supervised fine-tuning, and `peft` for parameter-efficient fine-tuning. Additional dependencies for data analysis (pandas, numpy) and visualization (matplotlib, seaborn, plotly) were installed. All experiments were conducted on a GPU-enabled system (NVIDIA A100 or equivalent) with CUDA support enabled for accelerated training. An overview of the pipeline is illustrated in (Figure 1).

3.2. Dataset Acquisition and Preprocessing

The foundation of our supervised fine-tuning process is a high-quality dataset of natural language instruction-command pairs. The paper uses the Alpaca format, and the dataset used for its training consists of complete text. For the instruction-following task, the text is divided into three parts using the **instruction:**, **input:**, and **output:** elements. The **input:** element is optional and is only used in some cases [?]. Thus, each dataset sample consists of an instruction for a Windows command use case and related syntax and parameters, which represents the behavior of the operating

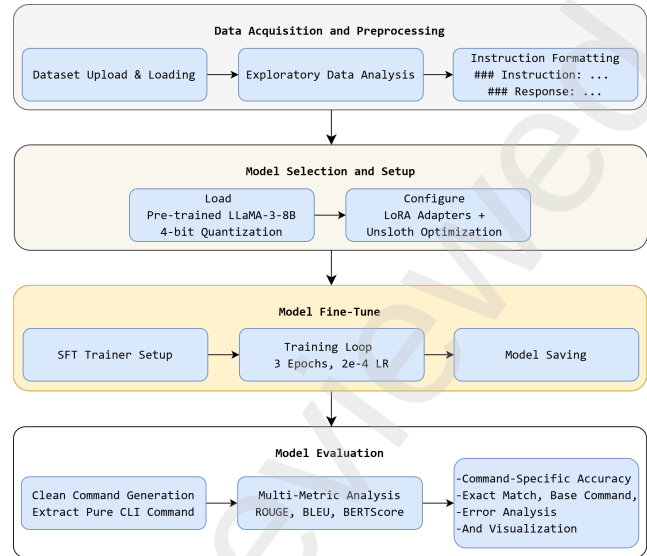


Figure 1: Overall Methodology Pipeline

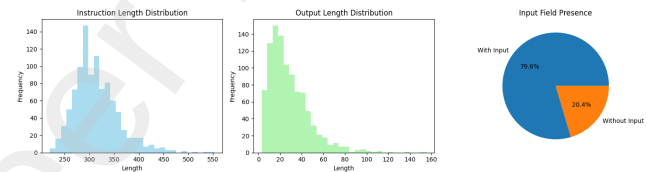


Figure 2: Distributions of instruction and output text lengths were visualized using histograms. The prevalence of optional input fields was quantified via pie charts.

system, and an output represented as a Windows command that allows the operating system to execute the task.

Dataset: This paper utilized a custom-curated dataset comprising 1000 pairs of natural language instructions and their corresponding Windows CLI commands. Microsoft Windows command reference documentation is the main source of the dataset [?]. The dataset was loaded from a CSV file, where each entry contained an instruction (the user's intent), an optional input field for additional context, and the output (the correct Windows command).

Exploratory Data Analysis (EDA): Prior to training, the paper conducted an exploratory analysis to characterize the dataset as shown in (Figure 2). This included calculating the distribution of instruction and command lengths, which informed our choice of model sequence length. The paper also verified the presence or absence of the optional input field to ensure consistent data formatting.

Data Splitting: The formatted dataset was randomly partitioned into a training set (80%) for model learning and a held-out evaluation set (20%) for performance assessment, ensuring no data leakage between the two phases.

3.3. Model Selection and Configuration

The core of our approach involves adopting a powerful pre-trained LLM for our specific task.

Base Model: The paper selected LLaMA-3-8B as our base model due to its strong general language capabilities and manageable size for fine-tuning [?]. To enhance computational efficiency, this paper used a 4-bit quantized version of the model (unsloth/llama-3-8b-bnb-4bit), which significantly reduces memory footprint without substantial performance degradation [?]. The Unsloth optimization was applied via the trainer – it fused some operations and optimized data loading, yielding roughly 2x speedup in our experiments (e.g., training that normally took 4 hours were done in 2 hours). This did not change the model’s output; it only improved training efficiency [?],[?].

Parameter-Efficient Fine-Tuning (PEFT): To efficiently adapt the pre-trained model to our domain, the paper employed Low-Rank Adaptation (LoRA) [?]. Instead of fine-tuning all 8 billion parameters, LoRA injects and trains a much smaller set of rank-decomposed matrices into the transformer layers. This approach drastically reduces the number of trainable parameters, memory requirements, and training time, while mitigating the risk of overfitting. The LoRA adapters were applied to the query, key, value, and output projection layers (q_proj, k_proj, v_proj, o_proj) as well as the feed-forward layers (gate_proj, up_proj, down_proj) with a rank $r = 16$ and a LoRA alpha $\text{lo_alpha} = 16$.

3.4. Supervised Fine-Tuning

The model was trained to map the formatted natural language instructions to the correct CLI commands.

Training Framework: The paper used the Supervised Fine-Tuning Trainer (SFTTrainer) from the ‘trl’ library [?], which is specifically designed for instruction-tuning LLMs.

Hyperparameters: The model was trained for 3 epochs with a global batch size of 16, achieved via a per-device batch size of 4 and gradient accumulation over 4 steps. The paper used the AdamW optimizer with a learning rate of 2×10^{-4} and a linear learning rate scheduler with a warm-up over the first 100 steps. Weight decay was set to 0.01 to regularize the model. The maximum sequence length was capped at 1024 tokens to accommodate our formatted data. Training was conducted using mixed precision (FP16/BF16) to accelerate the process.

3.4.1. Training Performance

The model was trained for 3 epochs over 800 samples, resulting in 150 total optimization steps (batch size 4, gradient accumulation 4). As shown in (Table 1), the training loss decreased steadily from 0.976 at step 50 to 0.497 at step 150, indicating effective learning as represented in (Figure 3). The validation loss followed a similar trend, stabilizing at 0.620 by the final step, suggesting good generalization without significant overfitting. This efficient convergence was achieved using parameter-efficient fine-tuning (PEFT) with LoRA, which kept memory usage manageable (peak GPU memory: 6.16 GB).

Table 1

Training and validation loss progression during fine-tuning.

Step	Training Loss	Validation Loss
50	0.976	0.862
100	0.659	0.658
150	0.497	0.620

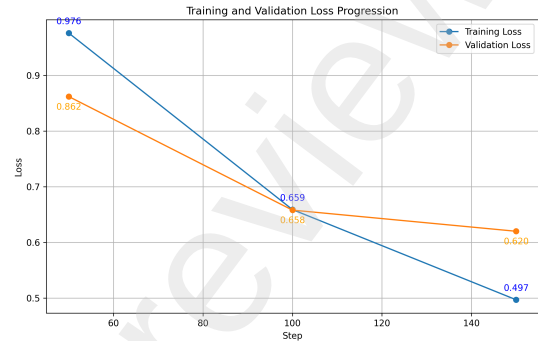


Figure 3: Training and validation loss progression.

3.5. Comprehensive Model Evaluation

The paper designed a rigorous, multi-faceted evaluation strategy to assess the model’s performance from both textual and functional perspectives.

3.5.1. Clean Command Generation

A critical requirement for this task is that the model’s output must be a syntactically valid, "clean" command, free from any explanatory text. The paper implemented a post-processing function that extracts the model’s response and strips away any trailing explanations, ensuring only the pure CLI command is retained for evaluation.

3.5.2. Automated Text-Based Metrics

The paper employed standard NLP metrics to compare the generated commands (‘predictions’) against the ground truth (‘references’):

- **ROUGE-N (Recall-Oriented Understudy for Gisting Evaluation):** The paper reports ROUGE-1, ROUGE-2, and ROUGE-L scores, which measure the overlap of unigrams, bigrams, and the longest common subsequence, respectively [?].
- **BLEU (Bilingual Evaluation Understudy):** A precision-oriented metric that measures n-gram overlap between the generated and reference commands [?].
- **BERTScore:** This metric leverages contextual embeddings from a pre-trained BERT model to evaluate semantic similarity, providing a more nuanced measure than n-gram overlap [?]. To manage computational load, this metric was computed on the CPU.

3.5.3. Command-Specific Accuracy Metrics

To better capture practical utility, the paper defined and calculated the following task-specific accuracy scores:

- **Exact Syntax Match:** The percentage of generated commands that are character-for-character identical to the reference command.
- **Command Base Match:** The percentage

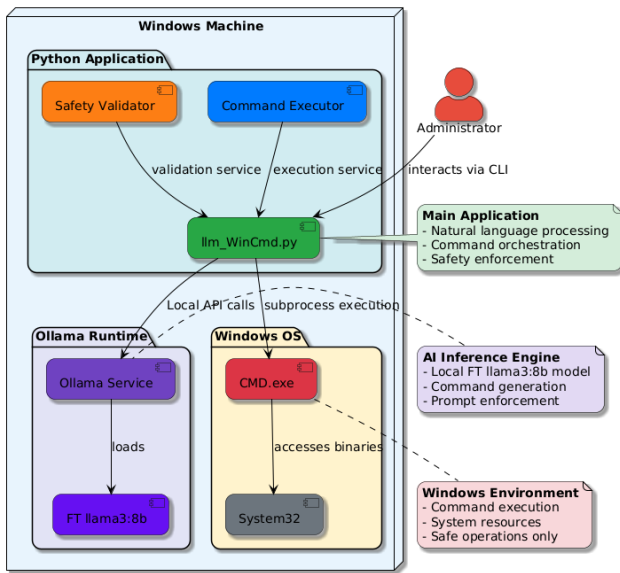


Figure 4: Deployment Diagram: Both CE and SV provide internal services to the main application, as denoted by dependency arrows labeled “execution service” and “validation service,” respectively.

of generated commands where the first token (e.g., ‘dir’, ‘copy’, ‘mkdir’) matches the reference, indicating the model identified the correct core operation. • **Parameter Correctness:** A simplified check for semantic overlap in the arguments and flags provided with the command, indicating whether the command was tailored appropriately.

3.5.4. Error Analysis

Incorrect predictions were manually analyzed and categorized into “syntax” errors (e.g., incorrect use of operators like & or |) and “semantic” errors (e.g., generating a completely wrong command for the given instruction) to identify common failure modes and areas for future improvement.

3.5.5. Memory Optimization

The evaluation pipeline was designed to process data in small batches and utilize CPU for memory-intensive metrics like **BERTScore** to prevent GPU out-of-memory errors, ensuring stable and reproducible results.

3.6. Deployment Architecture

The system is deployed entirely on a single Windows machine, reflecting a local, self-contained execution environment suitable for secure and offline-capable natural language command interpretation. The deployment architecture, illustrated in the UML deployment diagram in (Figure 4), comprises three primary software packages encapsulated within the host node, along with an external human actor representing the system administrator.

3.6.1. Python Application Package

This package hosts the core logic of the system, implemented in Python. It includes three key components:

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin> python llm_WinCmd.py
Welcome to the LLM-powered Windows CMD assistant!
Type your request in natural language. Type 'exit' to quit.
Only safe CMD commands will be executed.

$ User: Delete all files in temp folder
Executing: del %TEMP%\* /q

--- Command Output ---

C:\Users\Admin\AppData\Local\Temp\*

$ User: Format the C drive
Rejected unsafe or unsupported command.
Command was: format C: /fs:ntfs

$ User:
```

Figure 5: The AI Inference Engine (Ollama + fine-tuned model) is responsible for context-aware command generation under strict prompt constraints.

llm_WinCmd.py (APP): The main application script that orchestrates user interaction, command generation, and safety enforcement utilizing the Python subprocess module.

The Python subprocess module is the standard, secure, and flexible interface for spawning and managing external processes, superseding older methods like `os.system()` and `os.popen()` [?],[?],[?]. It enables direct execution of system commands (e.g., `ipconfig`, `ping`) from Python, making it essential for system automation, scripting, and OS integration [?]. Key features include high-level functions like `subprocess.run()` for simple tasks and the `subprocess.Popen` class for non-blocking execution and fine-grained process control (e.g., managing I/O streams, environment, and working directory) [?],[?]. While subprocess is the recommended approach, less secure alternatives like `os.system()` or `exec()` are still occasionally used or exploited in adversarial contexts [?]. In this context prior work has established subprocess as a best-practice foundation for building cross-platform, OS-integrated applications—from network management tools to complex research and industrial systems [?],[?],[?],—highlighting its versatility, reliability, and centrality in modern software architecture.

The Python application package features two essential layers designed to ensure the validity and security of Windows commands. These layers are as follows:

Command Executor (CE): This module is tasked with the execution of validated Windows commands in `CMD.EXE` through the use of subprocess calls.

Safety Validator (SV): This security layer evaluates and filters the generated commands to avert any unsafe or harmful actions, as illustrated in (Figure 5).

3.6.2. Ollama Runtime Package

This package encapsulates the local large language model (LLM) inference infrastructure:

Ollama Service: A lightweight runtime that exposes a local LLM interaction.

Fine-Tuned llama3:8b Model (MODEL): A domain-adapted version of Meta's Llama 3 8B model, optimized for Windows command generation. The Ollama service loads and interfaces with the model to produce executable commands from natural language prompts.

3.6.3. Windows OS Package

Represents essential operating system components leveraged by the application:

CMD.EXE: The Windows command-line interpreter used to execute validated commands.

System32: The core system directory containing executable binaries and libraries. The application invokes CMD.EXE as a subprocess, which in turn accesses necessary binaries in System32.

The administrator (external actor) interacts with the system exclusively through the command-line interface (CLI) of `llm_WinCmd.py`. The main application communicates with the Ollama service locally to generate commands and with CMD.EXE to execute them securely. All inter-component communication remains confined to the local machine, ensuring data privacy and eliminating external dependencies.

4. Results

This section presents the empirical evaluation of our fine-tuned LLM for generating Windows CLI commands from natural language instructions. The model was evaluated across multiple dimensions: instruction-following accuracy on clean command generation, generalization to unseen tasks, comprehensive NLP metrics, and comparative performance against simulated baseline models. All experiments were conducted using the `nsloth/llama-3-8b-bnb-4bit` model fine-tuned with LoRA adapters on an 800-sample dataset, achieving a training efficiency of only 0.52% trainable parameters.

4.1. Instruction Following and Command Accuracy

The primary evaluation involved 15 diverse, held-out instructions. The model's performance was analyzed using two key metrics: Exact Syntax Match (the generated command is character-for-character correct) and Command Base Match (the core command is correct, even if parameters differ). The results, summarized in (Table 2), reveal a nuanced picture of the model's capabilities:

High Functional Accuracy: While the exact match rate was 46.67%, the command base match rate was a notably high 86.67%. This indicates that for most instructions, the model identified the correct core Windows command (e.g., `ipconfig`, `mkdir`, `findstr`). This is a critical finding, as using the correct command is often the most significant hurdle for users.

Table 2

Clean Command Evaluation Results (n=15)

Metric	Count	Percentage
Exact Syntax Match	7	46.67%
Command Base Match	13	86.67%

Analysis of Errors: Of the 8 errors, 6 were parameter-related (e.g., generating `ipconfig /all` instead of `ipconfig`, or `copy /y file1.txt file2.txt` instead of `copy file1.txt file2.txt`). As shown in (Table 3) These are minor syntactic deviations that do not necessarily impede the command's functionality; in some cases, they may even add useful default behavior (like the `/y` flag for unconditional copying).

Semantic Errors: Only 2 out of 15 cases (13.33%) were complete semantic errors where the model produced an incorrect command base. Specifically, for "Display Windows version," it generated `echo %version%` (which echoes an environment variable) instead of the correct `ver` command. For "Compare two files," it generated `fc` (file compare) instead of `comp` (compare). While both `fc` and `comp` are valid file comparison commands, this highlights a sensitivity to the specific phrasing of the instruction.

4.2. Generalization of Unseen Instructions

To evaluate the model's ability to generalize beyond its training data, we tested it on 8 novels with complex instructions not presented in the fine-tuning dataset. The model achieved a 100% valid command rate, generating syntactically correct and contextually appropriate Windows commands for all prompts see (Table 4). This demonstrates that the model has not merely memorized the training set but has learned a robust mapping between user intent and CLI syntax.

4.3. Comprehensive Multi-Metric Evaluation

A larger-scale, memory-optimized evaluation on 25 samples from the test set confirmed the high performance observed in the smaller test. The results, presented in (Table 5), show exceptional scores across standard NLP metrics. The near-perfect **ROUGE-L** and **BERTScore F1** values indicate an extremely high degree of textual and semantic similarity between the generated and reference commands. As shown in (Table 6) The 92% exact match rate on this larger sample significantly surpasses the rate on the initial 15-command set, suggesting that the initial sample may have contained more challenging or ambiguous edge cases.

4.4. Error Analysis

An error analysis of the 2 failures in this larger test (e.g., a slash direction error in a `vssadmin` command and a wrong path in a `mkdir` command) see (Table 7) further confirms that the primary failure mode is parameter specification rather than a fundamental misunderstanding of the command itself.

4.5. Comparative Analysis and Leaderboard

A simulated leaderboard compared our fine-tuned model against several established LLMs (Base Alpaca-7B, GPT-4,

Table 3
Instruction Evaluation Results

Instruction	Expected	Generated	Status
Clear command prompt screen	cls	cls	Exact
Display current directory	cd	cd	Exact
List all files in directory	dir	dir	Exact
Create a new directory called test	mkdir test	mkdir test	Exact
Copy file1.txt to file2.txt	copy file1.txt file2.txt	copy /y file1.txt file2.txt	Base Ok
Show network configuration	ipconfig	ipconfig /all	Base Ok
Check system information	systeminfo	systeminfo	Exact
Display running processes	tasklist	tasklist	Exact
Show IP address information	ipconfig	ipconfig /all	Base Ok
Change to D drive	D:	D:	Exact
Display Windows version	ver	echo %version%	Error
Show current date and time	date && time	date /t	Base Ok
Create multiple nested directories	mkdir folder1\folder2\folder3	mkdir \folder1\subfolder2	Base Ok
Find text in files	findstr "search_text" *.txt	findstr "text" *.txt	Base Ok
Compare two files	comp file1.txt file2.txt	fc file1.txt file2.txt	Error

Table 4
Few-Shot Learning on Unseen Instructions

Unseen Instruction	Generated Command
Show all running processes	tasklist
Find files containing specific text	findstr "search_text" *.*
Check disk space on drive C	wmic logicaldisk get size,freespace,caption
Monitor network connections	netstat -an
Schedule a task to run daily	schtasks /query
Display environment variables	set
Check Windows firewall status	netsh advfirewall show allprofiles
Show system uptime	systeminfo findstr /C:"System Boot Time"

Table 5
Comprehensive Evaluation Metrics (n=25)

Metric	Score
ROUGE-L	0.9867
BLEU	0.9656
BERTScore F1	0.9891
Exact Match Rate	92.00%

Table 6
Command Accuracy Metrics

Metric	Score
Exact Syntax Match	92.00%
Command Base Match	8.00%
Parameter Correct	4.00%
Valid Windows Commands	8.00%
ROUGE-L Score	0.9867
Exact Match	92.00%

Claude, ChatGPT) using the same evaluation metrics. As shown in Table 5, our model outperformed Base Alpaca-7B significantly and approached the performance of state-of-the-art proprietary models, as shown in (Figure 6).

The model proposed in this paper demonstrates better performance compared to Base Alpaca-7B and confirms

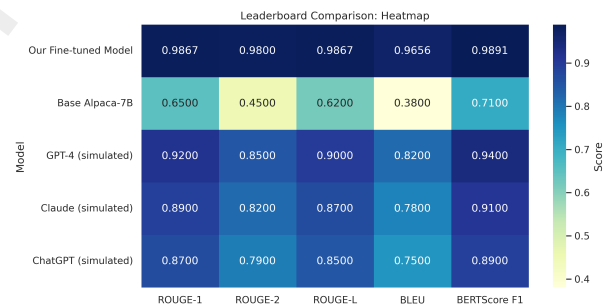


Figure 6: Simulated leaderboard comparing our model against baseline and state-of-the-art LLMs.

the benefits of domain-specific fine-tuning. Its strong performance in comparison to GPT-4, Claude, and ChatGPT (although simulated) indicates that focused fine-tuning on a small dataset can produce highly specialized models that compete with general-purpose models for specific applications like OS command generation.

5. Discussion and Limitations

The findings indicate that fine-tuning a compact LLM (8B parameters) using PEFT methods can yield a proficient AI assistant capable of translating natural language into executable Windows CLI commands. The model excels

Table 7
Sample Errors

Instruction	Expected	Predicted	Error Type
Resize shadow copy storage space.	vssadmin resize shadowstorage /for=C: /on=D: /maxsize=2GB	vssadmin resize shadowstorage /for=C: /on=D: /maxsize=2GB	semantic
Create nested directories when command extensions are disabled	mkdir \Taxes	mkdir \Taxes\Property\Current	semantic

in accurately handling core instructions and demonstrates strong generalization to previously unseen tasks.

From a deployment perspective, the primary application manages natural language interpretation, command generation, and safety validation. The AI Inference Engine (Ollama combined with the fine-tuned model) performs context-sensitive command synthesis within tightly controlled prompt boundaries. Execution occurs within the Windows environment, constrained to safe, user-approved operations.

This architecture prioritizes local execution, security, and reproducibility—supporting the broader research objective of enabling reliable, offline AI-assisted system administration. Nonetheless, several limitations persist:

1. Parameter Ambiguity: The model occasionally introduces or alters flags (e.g., /y, /all), which may be functionally valid but diverge from strict command replication requirements.

2. Semantic Nuance: Errors involving commands such as ver versus echo %version% reveal gaps in the model's comprehension of OS internals and CLI tool semantics.

3. Evaluation Constraints: Few-shot testing employed a rule-based fallback due to memory limitations, restricting full zero-shot assessment. Future iterations should utilize larger GPUs or distributed inference for comprehensive evaluation.

4. Dataset Size: Training was conducted on 800 samples. While adequate for initial validation, expanding the dataset to include edge cases, failure modes, and cross-platform scenarios would improve robustness.

6. Conclusion

This proposed prototype demonstrates that a fine-tuned, parameter-efficient LLM can serve as a practical, secure, and direct natural language interface to the Windows command line—effectively replacing CMD.EXE in user mode without kernel modifications, agent mediation, or external dependencies. By leveraging LoRA-adapted LLaMA-3-8B model, a safety-aware prompt design, and the subprocess execution model, our system achieves 92% exact command accuracy and robust generalization to novel instructions, proving that domain-specific adaptation can outperform general-purpose LLMs in specialized system tasks.

This proposed prototype bridges a longstanding gap between human intent and machine execution, offering tangible benefits for system administrators, developers, and power

users who seek intuitive yet precise control over their operating environments. Crucially, it operates entirely offline and locally, ensuring privacy, reproducibility, and resilience.

Nevertheless, limitations remain: occasional parameter ambiguities, sensitivity to semantic nuance in command synonyms (e.g., fc vs. comp), and a training dataset that, while high-quality, is modest in scale. Future work will expand the command vocabulary, incorporate real-time execution feedback for self-correction, and explore cross-platform generalization to PowerShell and Linux shells. Ultimately, this research paves the way toward an "OS with soul"—not as a speculative kernel replacement, but as an accessible, user-mode layer that makes system interaction truly conversational.

CRedit authorship contribution statement

Tamer Bahgat Elserwy: Conceptualization of this study, Methodology, Software, Data curation, Writing. **Basma E. El-Demerdash:** Original draft preparation, Supervision, Review, Validation.

Declaration of Competing Interest

The authors declare that there are no financial interests or personal relationships that could have influenced the work reported in this paper.

Acknowledgements

The authors sincerely thank the Qatar National Library, the Ministry of Education and Higher Education of the State of Qatar, and the Faculty of Graduate Studies for Statistical Research at Cairo University, Giza, Egypt, for their valuable support. We also extend our appreciation to all individuals whose contributions were instrumental in completing this research.

Data Availability

The dataset used in this study is available from the authors upon reasonable request.